

ams are working in separate groups within a single application, there is a high potential for our changes to impact other groups or the application as a whole. We have to be cautious not to inadvertently impact overall system quality but also availability, reliability, performance, and security.

An example would be a change to user authentication or login, which might impact seemingly unrelated services, such as project management or viewing an issue.

Far-reaching work is work that has wide-ranging, diffuse implications, and includes changes to areas which will:

1. be utilized by a high percentage of users
1. impact entire services
1. touch multiple areas of the application
1. potentially have legal, security, or compliance consequences
1. potentially impact revenue

If your group, product area, feature, or merge request fits within one of the descriptions above, you must seek to understand your impact and how to reduce it. When releasing far-reaching work, use a rollout plan. You might additionally need to consider creating a one-off process for those types of changes, such as:

- Creating a rollout plan procedure
 - Consider how to reduce the risk in your rollout plan
 - Document how to monitor the rollout while in progress
 - Describe the metrics you will use to determine the success of the rollout
 - Account for different states of data during rollout, such as cached data or data that was in a previously valid state
- Requiring feature flag usage (example)
- Changing a recommended process to a required process for this change, such as a domain expert review
- Requesting manual testing of the work before approval

Identified areas

Some areas have already been identified that meet the definition above, and may consider altered approaches in their work:

Area	Reason	Special workflows (if any)
Database migrations, tooling, complex queries, metrics impact to entire application		
The database is a critical component where any severe degradation or outage leads to an S1 incident. Documentation		
Sidekiq changes (adding or removing workers, renaming queues, changing arguments, changing profile of work required) impact to multiple services		
Sidekiq shards run groups of workers based on their profile of work, eg memory-bound. If a worker fails poorly, it has the potential to halt all work on that shard. Documentation		
Redis changes impact to multiple services		
Redis instances are responsible for		

sets of data that are not grouped by feature category. If one set of data is misconfigured, that Redis instance may fail. |

| Package product areas | high percentage of traffic share |

| Gitally product areas | high percentage of traffic share |

| Create: Source Code product areas | high percentage of traffic share. Special attention should be paid to Protected Branches, CODEOWNERS, MR Approvals, Git LFS, Workhorse and the git over SSH / gitlab-sshd interfaces. Please contact the EM (@seancarroll) or PM (@tlinz) if you are unsure. | |

| Pipeline Execution product areas | high percentage of traffic share | Documentation

| Authentication and Authorization product areas | touch multiple areas of the application | Documentation

| Compliance product areas | potentially have legal, security, or compliance consequences | Code Review Documentation

| Workspace product areas | touch multiple areas of the application | Documentation

| Specific fulfillment product areas | potentially impact revenue |

| Runtime language updates | impacts to multiple services | Ruby Upgrade Guidelines |

| Application framework updates | impacts to multiple services | Rails Upgrade Guidelines

| Navigation | impact to entire application | Proposing a change that impacts navigation

|

SECTION: engineering/devops/performance-indicators.md

{{% performance-indicators "devopsdepartment" %}}

SECTION: engineering/devops/dev/_index.md

SECTION: engineering/devops/dev/create/_index.md

Hello

We're the Create Stage and we are a group of teams within the Dev Sub Department. We're comprised of three areas within the GitLab product.

| Team | Engineering Managers |

| ---- | ----- |

Create Stage	Darva Satcher (Director, Engineering)
Create:Code Review	François Rosé (Backend) André Luís (Frontend)
Create:Remote Development	Adebayo Adesanya (Fullstack)
Create:Source Code	André Luís (Frontend) (Backend - Acting)

Mission

What do we do?

The Create Stage supports software development teams accelerate their delivery and reduce cycle times, by improving efficiency and collaboration. Our stage provides tools that support the beginning of the DevOps Life Cycle.

Whom do we serve?

We build tools for:

- Software Developers
- DevOps Engineers
- Development Team leads
- Product Managers
- Product Designers

How do we serve them?

The Product Vision Handbook Page provides specific examples for how we serve each type of GitLab user.

Vision

The following areas have been defined as our direction for the year:

- Fast, reliable, and easily managed Git storage
- Binary file workflows
- Delightful code review
- Better collaboration in OSS and within large organizations across many teams
- Unifying our editing experiences
- Better integrations

Professional Development

Each team member is encouraged to take positive steps towards improving their skills and knowledge.

Growing your skillset will lead to more insightful contributions to the GitLab Product. As your expertise grows, your impact on the product and the company also grows. Here at GitLab, it is important to us to invest in the Professional Development of our team members.

If there is an area you are interested in learning more about, please reach out to your manager so they can provide an environment that will allow you to grow in your areas of preference.

Some recommended resources are below:

Engineering Managers

- Recommended Books
- Training Materials

Individual Contributors

- Professional Development
- Recommended Books
- Training Materials
- Transitioning from an Individual Contributor to a Manager
- Skip-level Meetings
- Iteration

How we work

Each team work in the manner that best meets the needs of their product and their team.

- Create Engineering Managers
- Remote Development Team
- Code Review Team
- Source Code Team Backend, Frontend

Templates

We use templates in order to make our processes more transparent and efficient. Documenting practices once and reusing them often provides guidance and support throughout the stage.

How we work across teams

The Create Stage teams work together and play together. We are fortunate to be able to rely on each other to support and complement our features. Some examples of Cross Team Collaboration:

- The Source Code and Gitaly Teams often collaborate together on issues
- Gitaly Team Members have mentored Source Code team members on the Go Programming language

Every quarter we participate in a cross team bonding activity, Create Team Day.

How we live our values

Engineering Managers live our values every day.

[Read More about how Engineering Managers live GitLab Values](#)

How we measure Results

- OKRs
- Dashboards

- Issue Boards
- Monitoring

How we measure Iteration

- Logs
- MR Rate

How we track productivity

In an effort to drive further clarity and align expectations on what is expected for each role, the Create stage has expanded on Engineering performance indicators such as the MR Rate (above 12 per month) and will further specify expectations for each seniority level.

Why?

During talent assessments, Engineering Managers take a holistic approach to evaluate team members' performance. They consider a diverse set of interconnected metrics across our ecosystem, ensuring fair assessment against the expectations for each Job Grade. These metrics include:

- Merge requests merged
- Code reviews conducted
- Maintainer and interviewer status
- Number of interviews participated in
- Mentoring activities
- Merge Request Impact
- Multiple Others

This comprehensive approach allows for a well-rounded evaluation of an engineer's contributions and growth. For more details, refer to our [Talent Assessments page](#).

We believe it's important to communicate expectations clearly and transparently so we can all know what is expected and what to expect. This section looks to add clarity to this topic and promote higher awareness among the entire Create stage.

What metrics do we focus?

The goal of any team performance metric is to contribute partially to the understanding of a reality. This is because each metric might be imperfect on its own but alongside others can offer a more complete picture and better insights into the contributions given by the team and/or individual.

Currently we're setting clear monthly expectations for the following metrics:

- Merge Request Rate:
 - When applied to a group: The numerator is the number of merge requests merged into a set of projects (see notes below). The denominator is the number of people in the group.
 - When applied to an individual: It's the number of merge requests merged into a set of projects (see notes below).

- Reviews Rate:

- When applied to a group: The numerator is the number of code reviews given to merge requests that merged into a set of projects (see notes below) in a given period (usually a month). The denominator is the number of people in the group.

- When applied to an individual: It's the number of code reviews given to merge requests that merged into a set of projects (see notes below) in a given period (usually a month).

Dashboards

- Merge Request Rate: tableau dashboard

- Reviews Rate: tableau dashboard

Note: If you don't have access to Tableau, reach out to your direct manager to either provide you with a screenshot for the desired period or if applicable they can open an access request to provide you with permanent access.

Notes: {metricsnotes}

1. These metrics include all MRs affecting the product. The specific projects included in the dataset are listed in this seed file.

1. We will iterate on this process over time by expanding our metric set and refining them to ensure alignment with team contributions and evolving role expectations. Any changes will be clearly communicated to all team members.

Baseline targets for each job grade

In the table below, we outline the baseline numbers for each of the metric related to the Seniority level:

Metric	Associate	Intermediate	Senior	Staff
MR Rate	5	5	8	13
Review Rate	3	10	16	16

How were the targets calculated?

By analysing the existing teams' metrics and collaborating with all Engineering Managers in the stage, targets were defined to accurately reflect the expectations set during a normal calibration session for talent assessments at the Performing level of each role.

These targets were set having adherence to our CREDIT values in mind and are both ambitious and realistic.

What do the targets mean?

We expect team members to be on or above target for at least 6 out of the 12 months of a given year.

For Individual Contributors, these targets can provide a health status signal in alignment with expectations for the role and seniority they're in.

For Engineering Managers, these targets can provide a signal to review team processes, planning, or individual cases to better support their team members.

For any teams that have specific contexts that justify a deviation, that extra context should be documented in their handbook team page as much as possible.

SECTION: engineering/devops/dev/create/.gitlab/issue_templates/initiative.md

Overview

Description

Provide a concise description that outlines why the initiative exists (purpose), what specific achievements it aims for (goals), and why it holds significance or value (importance). The goal is to communicate the initiative's essence, objectives, and the broader impact it seeks to achieve.

Stakeholders

Stakeholders, external entities like Customer Success, Customer Support, and Marketing Leads, provide input and represent broader business needs. In contrast, the Team, comprising internal members such as Tech Lead, Engineers, and Product Managers, is actively engaged in executing tasks and achieving the goals of the initiative. Recognizing the distinction is crucial for clear communication, avoiding role overlap, and fostering effective collaboration as stakeholders guide the initiative's strategic direction, while the Team focuses on hands-on implementation, contributing to the initiative's overall success.

Objectives

This initiative's objectives are tied to GitLab OKRs (Objectives and Key Results) through the alignment of this initiative's goals with broader organizational objectives. GitLab's OKR framework typically includes high-level objectives that align with the company's strategic priorities. These objectives, on the other hand, are more granular and are designed to contribute to the achievement of these higher-level OKRs. Each Initiative should have at least 1 related OKR.

OKRs (Objectives and Key Results)

<!-- markdownlint-disable MD042 -->

1. OKR title goes here (link to current OKR for the quarter)
2. OKR title goes here (link to current OKR for the quarter)
3. OKR title goes here (link to current OKR for the quarter)

Team

Roles and Responsibilities

List team members along with their specific roles and responsibilities, establishing

accountability and coordination.

Team Member	Role	Responsibilities
Team Member 1	Tech Lead	Technical vision, guidance, mentoring
Team Member 2	Engineer	Code implementation, testing
Team Member 3	UX Designer	User experience design, reviews
Team Member 4	Product Manager	Milestone planning, coordination, reporting
Team Member 5	SRE	Ensuring the reliability and performance of the system

Exit Criteria

Once all the exit criteria are fulfilled, the respective initiative will be concluded and closed out.

Completed	Exit Criteria	Start Date	Completion Date
[]	All features implemented	January 15, 2025	February 20, 2025
[]	Code review completed	February 25, 2025	March 5, 2025
[]	Unit tests passed	March 10, 2025	March 15, 2025
[]	System tests passed	March 20, 2025	March 25, 2025
[]	Documentation updated	March 30, 2025	April 5, 2025
[]	User acceptance testing done	April 10, 2025	April 15, 2025
[]	Performance benchmarks met	April 20, 2025	April 25, 2025
[]	Security review completed	May 1, 2025	May 5, 2025
[]	Stakeholder approval obtained	May 10, 2025	May 15, 2025

Timeline

Start Date	Expected End Date	Actual End Date	Status (Not Started, In Progress, Complete)	Epic [Link to Epic]	Issue [Link to Issue]
Assignee Mention					
March 1, 2025	-	-	In Progress	-	-
Define scope	@[Assignee 1]				
March 15, 2025	March 30, 2025	-	Not Started	User Authentication	Design User Interface
March 30, 2025	April 15, 2025	-	Not Started	Security Audit	Address Identified Vulnerabilities
April 15, 2025	April 30, 2025	-	Not Started	Database Queries Optimization	Test and Measure Performance
April 30, 2025	May 10, 2025	-	Not Started	-	-
User Acceptance Testing	@[Assignee 5]				
May 10, 2025	May 20, 2025	-	Not Started	-	-
Ensure All Issues Resolved	@[Assignee 6]				

Risk Management

Identify potential risks and strategies for mitigation, preparing the team for unforeseen challenges.

Risk Description	Likelihood	Impact	Mitigation Plan
Potential scope creep	Medium	High	Regular scope reviews and communication with stakeholders to manage expectations.
Key team member availability	Low	High	Cross-training team members to ensure knowledge sharing and having backup plans for critical roles.
Technical dependencies on third-party tools	High	Medium	Continuous monitoring of third-party tools, and having alternative solutions in place.

Communication Plan

Effective Communication Channels

- Team Meetings:
 - Regularly scheduled meetings to discuss progress, challenges, and upcoming tasks.
 - Frequency: Weekly on Mondays at 10 AM UTC.
 - Agenda: Link to Doc.
- Slack Channel:
 - Dedicated Slack channel for ad hoc communication, quick questions, and announcements.
 - Team members are encouraged to use @mentions for specific communication.
- Status Report Issues:
 - Create an issue in the project repository for reporting status updates, achievements, and challenges. You can create either one issue (1, 2) that is updated at a specific cadence or different issues for every update (1,2,3). Some people prefer to post status reports in Slack, however, the problem with that is that you lose the history in Slack and it is not easy to search.
 - The preference is to use get with the product and dog food the product and then you can link from Slack to the GitLab issue.
 - Link to Issue Board / Issue Lists
 - Link to Issue Lists
 - Feedback Gathering Epic:
 - Link to an epic dedicated to gathering internal and external feedback.
 - Regularly check for feedback on features, user experience, and overall progress.

Metrics/Dashboards/Charts

Metrics

1. Tableau Chart 1
1. Tableau Chart 1
1. Tableau Chart 1

Monitoring

1. Grafana Board 1
1. Kibana Graph 1

Labels

Establish a labeling system for improved organization and tracking.

Label Name	Description	When to Apply
Applied By		
----- ----- -----		
----- -----		
Enhancement	Marks tasks for feature improvements	Prior to starting feature development
Product Team		
Critical	Identifies critical issues or bugs	When a critical issue is reported
Developers		
Review Pending	Indicates code or work awaiting review	After completing a task or feature
Developers		
Ready for Deploy	Signifies tasks ready for deployment	After successful testing and review
DevOps Team		
Documentation	Flags tasks requiring documentation	Throughout the development process
Developers		

Remember, the specific labels and their usage may vary depending on your initiative's workflow and requirements. It's essential to tailor them to fit the needs and processes of your team.

Quality, Security, SRE, Technical Writers

Create separate Epics for Quality, Security, SRE, TW or any other counterparts you need to complete the effort, ensuring focused collaboration in each area.

Start Date	Expected End Date	Actual End Date	Counterpart	
Epic Title		Exit Criteria		
----- ----- ----- ----- -----				
----- ----- ----- -----				
January 15, 2025	February 20, 2025	March 5, 2025	Quality	Enhance and ensure the overall quality of the product through rigorous testing and QA processes. Achieved desired test coverage, no critical bugs, and improved code quality.
February 1, 2025	March 15, 2025	March 25, 2025	Security	Implement and strengthen security measures to safeguard the integrity and confidentiality of the system. Implemented required security protocols, conducted successful penetration tests.
March 1, 2025	April 5, 2025	April 15, 2025	SRE	Focus on Site Reliability Engineering (SRE) practices to enhance system stability, performance, and availability. Achieved agreed-upon system reliability metrics and improved incident response.
April 1, 2025	May 5, 2025	May 15, 2025	Documentation	Improve and maintain comprehensive documentation for better understanding and ease of collaboration. Updated documentation with new features, clear guidelines, and improved user manuals.

<!-- markdownlint-enable MD042 -->

Feedback

Feedback Gathering

Set up an Epic for both internal and external feedback, promoting continuous improvement.

Project Team Meetings

Regular Meetings

- Meeting Agenda
- Meeting Recordings

Retrospective

Add a link to the retrospective issue for this initiative here. All initiatives should have a retrospective issue.

SECTION: engineering/devops/dev/create/code-review/_index.md

The Create:Code Review Group is responsible for all aspects of the product categories that fall under the Code Review group of the Create stage of the DevOps lifecycle.

Group overview

Group members

The following people are permanent members of the Create:Code Review Group:

{{<team-by-departments "Create:Code Review" >}}

Sub-department specific pages

- Backend
- Frontend

Product categories

The Code Review group is responsible for the following product categories:

- Code Review
- GitLab CLI

Category performance indicators

- Code Review Category MAU (Internal Only)
- Editor Extension Category MAU (Internal Only)

Work

In general, we use the standard GitLab engineering workflow. To get in touch with the Create:Code Review team, it's best to create an issue in the relevant project (typically GitLab) and add the ~"group::code review" label, along with any other appropriate labels (~devops::create, ~section::dev). Then, feel free to ping the relevant Product Manager and/or Engineering Manager as listed above.

For more urgent items, feel free to use gcreatecodereview on Slack.

Work on the GitLab VS Code Extension follows a simplified development process. Learn more about it by looking at CONTRIBUTING.md.

Take a look at the features we support per category here.

Metrics of success

The metrics by which we measure the success of the Code Review category are aligned with our goals for code review, specifically ease of use, love-ability, and efficiency.

Primary metric

Our primary metric is: reducing the duration of the Code Review. This is measured as the duration from the first merge request version to merged.

The MTTM can be found on this dashboard.

Secondary metrics

Secondary metrics of success act as support for the primary metric, helping build a more complete picture of how successful the category is.

Once in a while, we conduct UX scorecards to track the user experience through various heuristics - see all UX scorecards for Code Review. At the Create stage level, we conduct usability benchmarking studies.

Right now we're focused on measuring and improving perceived performance: "how fast, responsive, and reliable a website feels to its users. The perception of how well a site is performing can have more impact on the user experience than the actual load and response times." Perceived performance is not only technical performance (i.e. load and response times), but also user performance (i.e. efficiency in completing tasks), and can be formulated as:

text

perceived performance = f(expected performance, UX, actual performance)

experience = f(perceived performance, task completion)

Aspect	Measured by	Results
--------	-------------	---------

- -		
-------	--	--

Expected performance and UX	Primarily by user's feedback, and secondarily by actual	
-----------------------------	---	--

performance of competitors. | SaaS user's feedback (in progress)
Competitor performance (Software Forge Performance Index) (maintained by SourceHut)
Largest Contentful Paint of SaaS vs GitHub.com for key pages |
| Actual performance (load and response times) | Primarily by the Largest Contentful Paint (LCP) metric, and secondarily by other important metrics. | Test instance (test samples: large MR overview and changes tabs, large MR commits tab)
SaaS: gitlab-foss large MR overview tab (test sample)
SaaS: gitlab-foss large MR changes tab (test sample)
SaaS: gitlab-foss empty MR overview tab (test sample)
SaaS: gitlab large MR overview tab (test sample)
SaaS: gitlab small MR overview tab (test sample)
SaaS: Other project MR overview tab (test sample) |
| Task completion (task times) | Estimates of user's execution time of primary tasks through the GOMS approach. We focus on the percentage difference of GitLab and competitors, or of current and proposed designs. | July 2021 estimates |

Exploration and experimentation

The Code Review group believes it's important that team members are empowered to explore and experiment with areas of the product that interest them. Sometimes, the best way to get the conversation started is with a merge request.

Allocate time

In order to better provide opportunities for team members to pursue interest areas, engineers are encouraged to reserve about 10% of their scheduled capacity.

Setting expectations

If you're choosing to work in these areas or explore new ideas, there's a few ground rules to make sure we're all on the same page:

1. Work in these areas doesn't come at the cost of planned deliverables for the milestone
1. Not all efforts in these areas will be merged into the product, but sharing them with product and design can help steer conversations for future work
1. Work does not need to be in the code review area; engineers are encouraged to explore areas of interest

Areas of inspiration

It can be hard to figure out where to get started, so here's a handy list of places you might look for inspiration:

1. Top level Code Review epic - Epics in this list are loosely sorted by importance
1. Top level Editor Extension epic - Epics in this list encompass an entire category of features, but the group is primarily focused only on VS Code
1. Group level gitlab-org issue list - filter this to issues with labels you're interested in
1. Code review issues ready for development
1. Performance and Performance Refinement issues
1. Easy wins

Meetings

Whenever possible, we prefer to communicate asynchronously using issues, merge requests, and Slack. However, face-to-face meetings are useful to establish personal connection and to address items that would be more efficiently discussed synchronously such as blockers.

We record our meetings and upload them to the Create Code Review Playlist on GitLab Unfiltered.

Code Review Weekly

This is a chance for all members of the Code Review Group to meet to discuss current priorities, blockers, planning, and discuss the middle of milestone check-in.

The agenda for this meeting is set in advance and anyone can contribute topics. If there are no items on the agenda 30 minutes before the meeting is scheduled to start, we cancel the meeting.

Code Review UX Sync

This meeting is focused on collaboration between UX and PM, though all are welcome to attend and contribute.

Code Review Performance Round Table

This meeting is for discussing new performance topics, project proposals, and ongoing performance work or concerns.

The catch-all issue includes the agenda, issue board, and goals.

Working with product

Weekly calls between the product manager and engineering managers (frontend and backend) are listed in the "Code Review Group" calendar. Everyone is welcome to join and these calls are used to discuss any roadblocks, concerns, status updates, deliverables, or other thoughts that impact the group. Every 2 weeks (in the middle of a release), a mid-milestone check-in occurs, to report on the current status of ~"Deliverable"s. Monthly calls occurs under the same calendar where the entire group is encouraged to join, in order to highlight accomplishments/improvements, discuss future iterations, review retrospective concerns and action items, and any other general items that impact the group.

Collaborating with other counter parts

You are encouraged to work as closely as needed with stable counterparts outside of the PM. We specifically include quality engineering and application security counterparts prior to a release kickoff and as-needed during code reviews or issue concerns.

Quality engineering is included in our workflow via the Quad Planning Process.

Application Security will be involved in our workflow at the same time that kickoff emails are sent to the team, so that they are able to review the upcoming milestone work, and notate any concerns or potential risks that we should be aware of.

Working with the wider GitLab community

Since we support such a large feature set, our team often reviews community contributions from the wider GitLab community. You're encouraged to give each contributor our version of "white glove treatment". Providing recognition for their donated time, giving exceedingly helpful reviews, and encouraging them in their contribution are all excellent ways to build a sense of community. If you don't have time to respond to a ping for a review or suggestion, please quickly let the person who pinged you know so they can ping someone else.

Tips and Tricks

For tips, tricks, or quick shell scripts that aren't "ready" or sufficient enough to add to our developer documentation or handbook, we use the Create stage wiki.

Middle of milestone check-in

```
{{% include "includes/engineering/midmilestone.md" %}}
```

Workflow labels

```
{{% engineering/workflow-labels "group::code review" %}}
```

Async standup

```
{{% include "includes/engineering/create-async-standup.md" %}}
```

Our team is encouraged to post links to their deliverable issues or merge requests when they are mentioned in relation to the second question. This helps other team members to understand what others are working on, and in the case that they come across something similar in the future, they have a good reference point.

Retrospectives

We have 1 regularly scheduled "Per Milestone" retrospective, and can have ad-hoc "Per Feature" retrospectives more focused at analyzing a specific case, usually looking into the Iteration approach.

Per Milestone

```
{{% engineering/create-retrospectives group-label="Code Review" group-slug="code-review" %}}
```

Per Project

If a particular issue, feature, or other sort of project turns into a particularly useful learning experience, we may hold a synchronous or asynchronous retrospective to learn from it. If you feel like something you're working on deserves a retrospective:

1. Create an issue explaining why you want to have a retrospective and indicate whether this should be synchronous or asynchronous
1. Include your EM and anyone else who should be involved (PM, counterparts, etc)
1. Coordinate a synchronous meeting if applicable

All feedback from the retrospective should ultimately end up in the issue for reference purposes.

Merge Request Report Widgets shared responsibility

Even though the topic of Merge Request falls under Code Review, the code powering the Merge Request Report Widgets (see Working Group) is written and maintained by a larger group.

Please refer to the List of DRIs for communication and troubleshooting purposes relating to these Widgets.

SECTION: engineering/devops/dev/create/code-review/frontend.md

Common Links

Category	Handle
-----	-----
GitLab Team Handle	@code-review-fe
Slack Channel	gcreatesource-code-review-fe
Slack Handle	@codereviewfe
Team Boards	Current Milestone
Issue Tracker	group::code review + frontend in gitlab-org

Team Vision

A central piece in GitLab users' experience, innovating and keeping the experience delightful for all product categories that fall under the Code Review group of the Create stage of the DevOps lifecycle.

Team Mission

Support all our counterparts with frontend engineering expertise, including implementation, tech debt management, and timely frontend insights in discovery phases.

Commonly Monitored Issue Lists

Code Review + Frontend issues

Milestone Planning Issues

Triage reports

Feature flag reports

OKRs (confidential)

Team Members

The following people are permanent members of the Create:Code Review FE Team:

```
{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Code Review"
team=".(Frontend|Fullstack).Create:Code Review" >}}
```


Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts manager-role="Senior Engineering Manager(.)Create:Code Review" role="(Product Manager|Technical Writer|Software Engineer in Test|Senior Security Engineer|Product Designer).(Create:Code Review|Create \((Code)|Dev\|Create|Code Review" >}}
```

Core Responsibilities

Collaborate with Product and UX on ideation, refinement and scheduling of relevant work

Provide Frontend support for feature development, bug fixes, under the Code Review Workflow

Product Category

Address bug reports and regressions

Identify and prepare maintenance work to improve developer experience

Collaborate on efforts across the Frontend department

Projects

Active Project Table

Start Date	Project	Description	Tech Lead	
-----	-----	-----	-----	
2019	Merge Requests	Vue app	The frontend application that renders Merge Requests	·
2023-09	New Diffs (Epic)	A project to deliver a reusable and performant way of rendering diffs across GitLab	·	

Engineering Onboarding

Work

See the work section of the main Code Review page.

Capacity planning

```
{{% include "includes/engineering/create/weights-fe.md" %}}
```

Kickoff & Retrospective Review

On the first week of every milestone, we have a sync-call with every IC in which we review the comments shared in the async retrospective and after that each engineer takes turns at presenting their plan for their Deliverables for the new milestone.

This is scheduled ad-hoc within the first 5-10 days of the milestone.

Other related pages

Issues

2020 April: Frontend: Iteration Retrospective (Source Code)

2020 December: Merge Request Architecture Walkthrough

SECTION: engineering/devops/dev/create/code-review/report-widgets-dri-list.md

Merge Request Report Widgets - DRI list

In the context of the Working Group, a list of DRIs have been identified to facilitate communication among all Widgets currently active and in development.

Please refer to this list and keep it updated as Widgets evolve.

Widget	UX issue	Stage/Group	DRI (UX)	DRI (Engineering)
-----	-----	-----	-----	-----
(Core) / Global Feature Flag Rollout	&6647	Create:Code Review	@jeldergl	@iamphill
Accessibility	336180	Verify:Pipeline Security	@gdoyle	@mfluharty
Browser performance	336179	Verify:Pipeline Security	@gdoyle	@mfluharty
Code quality	336178	Secure:Static Analysis	@mfangman	@janniklehmann
License compliance	336177	Secure:Composition Analysis	@andyvolpe	@farias-gl
Metrics	336176	Verify:Pipeline Security	@gdoyle	@mfluharty
Security	336175	Security Risk Management:Threat Insights	@beckalippert	@svedova
Terraform	336174	Configure	@andlovu	@annavovchenko
Test summary	336173	Verify:Pipeline Security	@gdoyle	@mfluharty
Status checks	336172	Software Supply Chain Security:Compliance	@cam.x	@xanf
Load performance	336448	Verify:Pipeline Security	@gdoyle	@mfluharty

Related links

Merge Request Extensions technical documentation
Merge Request Report Widget Working Group page

SECTION: engineering/devops/dev/create/code-review/backend/_index.md

Team Vision & Mission

Please refer to the category directions for Code Review Workflow and GitLab CLI for the current strategy and 1 year plan for these products.

Core Responsibilities

The Create:Code Review BE team is responsible for all backend aspects of the product categories that fall under the Code Review group of the Create stage of the DevOps lifecycle:

- Code Review Workflow, mainly the Merge Requests features
- GitLab CLI

Group Members

The following people are permanent members of the Create:Code Review Group:

```
{{< team-by-manager-role role="Engineering Manager, Create:Code Review" team="Code Review" >}}
```

Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts role="Code Review" manager-role="Engineering Manager, Create:Code Review" >}}
```

Common Links

- GitLab Team Handle: @code-review-be
- Slack Channel: gcreatecode-review
- Slack Handle: @codereviewbe
- Issue Tracker: create-stage/code-review-be

Commonly Monitored Issue Lists

- Planning Issues
- OKRs
- Release Board
- Workflow Board (be sure to add a milestone filter)
- Reliability
- Security
- InfraDev Issues

Metrics and KPIs

These KPIs are a subset of the Development Department Performance Indicators.

You can find our dashboards here:

- Centralized Engineering Metrics
- Development Department Performance Indicators

Team OKRs

Objectives and Key Results (OKRs) help align our team towards what really matters. These happen quarterly, align up through the stage. We check in on the status of our progress routinely throughout the quarter, at least on a monthly basis, to determine whether we are on track or need to pivot in order to accomplish or change these goals.

For a list of current and past OKRs for our team, use this link.

How We Work

See the work section of the main Code Review page.

Working with our team

As stewards for some of the more high profile features - Merge Request pages, Approval Rules, etc - we receive a large number of inquiries and requests for assistance or information about them, as well as the dependent features that it encompasses. We welcome them, as issues or Slack inquiries, and we strive to be responsive to these, in the interest of Collaboration and Results, but we also must balance the value of Efficiency.

Our target is to respond to incoming requests within 2 working days, although we will frequently respond more quickly, depending on team member availability, experience, and workload.

To contact the Code Review BE team in GitLab, you can use the @code-review-be handle.

Meeting Schedule

Team members meet monthly and are encouraged to join in order to interact with peers, solve technical challenges, and discuss freely with each other. You can check for the next meeting on the Code Review Group calendar.

Follow-up issues

You will begin to collect follow-up issues when you've worked on something in a release but have tasks leftover, such as technical debt, feature flag rollouts or removals, or non-blocking work for the issue. For these, you can address them in at least 2 ways:

- Add an appropriate future milestone to the follow-up issue(s) with a weight and good description on the importance of working this issue
- Add the issue(s) to the relevant planning issue

You should generally take on follow-up work that is part of our definition of done, preferably in the same milestone as the original work, or the one immediately following. If this represents a substantial amount of work, bring it to your manager's attention, as it may affect scheduling decisions.

If there are many follow-up issues, consider creating an epic.

Backend and Frontend issues

Many issues require work on both the backend and frontend, but the weight of that work may not be the same. Since an issue can only have a single weight set on it, we use scoped labels instead when this is the case: ~backend-weight::<number> and ~frontend-weight::<number>.

What to work on

The primary source for things to work on is the Code Review Release board for the current iteration cycle, which lists all of the Deliverable and Stretch issues scheduled for this cycle.

The lists are compiled by the Product Manager and Engineering Managers following the

milestone planning process,
with input from the team and other stakeholders.
The iteration cycle start on the Monday before the 3rd Thursday of the month,
and is identified by the GitLab version set to be released.

There is also the Code Review backend assignment issue board,
which shows the same Deliverable
and Stretch issues, now grouped by assignee, with the left-most list listing
issues not currently assigned to any backend engineer.

What to work on first

Deliverables are considered top priority and are expected to be done by the end
of the iteration cycle on the Friday before the 3rd Thursday of the month,
in time for the monthly release.

These top priority issues are assigned to engineers on or ahead of the start of the
milestone, and it is their responsibility
to make a best effort to get them done during that cycle, and to inform their
engineering manager if anything is standing in the way of their success.
You can find the issues assigned to you on the Code Review backend assignment issue board.

Many things can happen during a month that can result in a deliverable
not actually being completed by the end of a cycle, and while this usually
indicates that the engineering manager was too optimistic in their estimation
of the issue's weight, or that an engineer's other responsibilities ended up
taking up more time than expected, this should never come as a surprise to the
engineering manager.

The sooner this potential outcome is anticipated and communicated, the more time
there is to see if anything can be done to prevent it, like reducing the scope
of the deliverable, or finding a different engineer who may have more time to
finish a deliverable that hasn't been started yet.
If this outcome cannot be averted and the deliverable ends up missing the
cycle, it will simply be moved to the next cycle to be finished up, and the
engineer and engineering manager will have a chance to
retrospect and learn from what happened.

Generally, your deliverables are expected to take up about 75% of the
time you spend working in a month. The other 25% is set aside for other
responsibilities (code review, community merge request coaching, helping people out in Slack,
participating in discussions in issues,
etc), as well as urgent issues that come up during the month and need someone
working on them immediately (regressions, security issues, customer issues, etc).

What to work on next

If you have time to spare after finishing your deliverables and other
activities, you can spend the remaining time working on Stretch issues, which
can be found on the same issue boards.

These lower priority issues are not expected to be done by the end of the iteration cycle, but are to be Deliverables in the next cycle, so any progress made on them ahead of time is a bonus.

Stretch issues are usually not directly assigned to people, except in cases where there is clearly a most appropriate person to work on them, like in the case of technical debt, bugs related to work someone did recently, or issues someone started on before but hasn't had a chance to finish yet.

If no Stretch issues are assigned to you yet, you can find new ones to pick up by filtering the Code Review Release board

If anything is blocking you from getting started on an issue immediately, like unanswered questions or unclear requirements, you can skip it and consider another issue, as long as you put your findings and questions in the issue, so that the next engineer who comes around may find it in a better state.

Instead of picking up Stretch issues, you may also choose to spend any spare time working on anything else that you believe will have a significant positive impact on the product or the company in general.

As the general guidelines state, "we recognize that inspiration is perishable, so if you're enthusiastic about something that generates great results in relatively little time feel free to work on that."

We expect people to be managers of one and prefer responsibility over rigidity, so there's no need to ask for permission if you decide to work on something that's not on the issue board, but please keep your other responsibilities in mind, and make sure that there is an issue, you are assigned to it, and consider sharing it in gcreatecode-review.

Deep Dives

```
{{% include "includes/engineering/create/deep-dives.md" %}}
```

Performance Monitoring

The Create:Code Review BE team is responsible for keeping some API endpoints and controller actions performant (e.g. below our target speed index).

There are Grafana and Kibana dashboards available that the team can check to ensure we are meeting these targets.

Here are the Kibana dashboards that give a quick overview on how they perform:

- Create::Code Review Controller

Actions%2CrefreshInterval%3A(pause%3A!t%2Cvalue%3A0)%2Ctime%3A(from%3Anow-7d%2Cto%3Anow)
(internal only)

- Create::Code Review: API

Endpoints%2CrefreshInterval%3A(pause%3A!t%2Cvalue%3A0)%2Ctime%3A(from%3Anow-7d%2Cto%3Anow)
(internal only)

- Create::Code Review: Sidekiq Workers (internal only)

These tables are filtered by the endpoints and controller actions that the group handles and sorted by P90 (slowest first) for the last 7 days by default.

The Grafana dashboard shows more specific details about each actions and endpoints.

To see the specific details for certain actions/endpoints, it can be filtered by action and/or controller.

We also have a dashboard specifically for monitoring AI features: Create: Code Review: AI Features%2Ctime%3A(from%3Anow-7d%2Cto%3Anow))) (internal only). This dashboard monitors the performance of code review AI features, specifically tracking P50 (median) duration for both Sidekiq and GraphQL operations.

Issue identification process

1. Every week we check the kibana and grafana dashboards as per the calendar invite
2. If we identify an endpoint or worker that meets our severity criteria we create an issue (if not created already) and label it based on our severity and priority criteria
 - If an issue is already created, check whether the severity/priority needs to be updated
3. This issue should contain as much information on why it is the severity/priority as logs are often held for only 7 days
4. Using a specific performance issue board we can keep track of our current issues and they will be prioritized accordingly during planning

Priority and Severity

More details can be found in the handbook for severity and priority

Calendar Invites

There are calendar invites that act as a reminder for backend team members to check these dashboards weekly on Monday:

- APAC: Check performance dashboards - set on Monday 12AM UTC
- AMER: Check performance dashboards - set on Monday 4PM UTC

All backend team members are invited to appropriate calendar invites. This does not necessarily mean that we only need to take a look at them at those specific times.

If a team member sees that an action or endpoint does not meet our target, they should create a performance issue if there's no existing one. If there's an existing one, team member can either update the issue, investigate or work on the issue (if they have spare time).

Responsibilities of Issue Triage

- Do a weekly review of the triage board and make sure these issues are investigated by assigning them to developers or performing the investigation
- Coordinate with Product Manager to get those issues assigned to milestones so that they can be worked on in timely manner
- Check issues in each column in the order of severity. ~missed-SLO -> ~approaching-SLO -> ~severity::1 -> ~severity::2 -> ~severity::3 -> ~severity::4
- Ensure the severity label is accurate based on the problem investigated
- Our aim is getting to 0 issues in ~missed-SLO column eventually and handle issues well before they get labeled as ~missed-SLO and ~approaching-SLO

Engineering Onboarding

Open an issue using the Code Review onboarding template for Backend Engineers to get started with onboarding.

Other Related Pages

- Create:Code Review BE Engineer Resources, e.g. team building and career development
- Create:Code Review BE Engineering Manager Responsibilities, e.g. milestone planning, talent assessment, and project management

SECTION: engineering/devops/dev/create/code-review/backend/engineers.md

This page contains engineering resources for the Create:Code Review Backend team. For day-to-day work of the team, please review the Create:Code Review BE handbook page.

Team building

Every two weeks a question/activity will be posted in the code-review-backend slack channel to enable the team to have a more relaxed interaction with each other. Currently, in the team, it is impossible to get everyone on a synchronous call and while the team days work well, they are at the create stage level. This team-building exercise is aimed more specifically at the Code Review Backend Team.

Some examples would be:

- Ice breaker style question: What is your favourite food to cook? (maybe share photos too), What was your favourite holiday and why?
- 'Gif battle': where we would have a topic, for example (cutest animal, most insane dance move) and we would each post our top GIF
- Share a photo: where we each share a photo that we took during the week

Feedback

It can be hard to understand how you're doing in your role, because feedback can come off as formal (annual reviews, 360 surveys, career development conversations, goal check-ins) or casual (in Slack channels, 1-1's, MR reviews, team meetings.) We receive various kinds of feedback regularly and through different formats, so the type of feedback you're receiving is

not always clear. In order to be more intentional about the types of feedback given, here is a classification chart based on three types of feedback:

Label	Meaning	Example
(appreciation)	I want to thank you for doing this, and please do more of it in the future	"I did not expect that you would have created a working group, because you've done so, our whole team will benefit from the results."
(coaching)	I'm trying to help you improve a behavior you are already exhibiting or change a behavior that you currently have	"The reports that you give me are very helpful, and in the future we can schedule them for the first of the month to be more consistent."
(evaluation)	Tells you where you stand according to existing standards or expectations	"My expectation was that our decision would be transparent. Since it was not, our team has forgotten the decision, so we must be sure and meet that expectation next time."

Career development

```
{{% engineering/create/career-development group="Code Review" %}}
```

Training and Development Opportunities

This is a list of commonly requested training and some additional resources:

1. The Database maintainer process centralizes training related to being not just a maintainer, but all the resources for a reviewer as well.
1. The Golang training template offers resources related to learning Go.
1. The feature flag training template will help get you started in understanding how to use feature flags at GitLab, as well as monitor the performance of your work.
1. There are 2 YouTube playlists that cover our monitoring and visualization tools. More resources are available in the handbook in terms of Grafana, Kibana, Prometheus, Sentry, and Sitespeed.io.
1. There is a DIB training template that utilizes internal resources and LinkedIn Learning, as well as an additional DIB certification course which is offered directly from GitLab.
1. The "transitioning to a manager" page and related training issue helps provide an idea of what management is like at GitLab.
1. Rails performance materials (internal only) are available from a workshop that was done at GitLab.

Additional resources

- Create Stage Professional Development
- Level Up
- Growth and Development Fund

```
## SECTION: engineering/devops/dev/create/code-review/backend/manager.md
```

The Backend Engineering Manager role at GitLab is an exciting and dynamic position that encompasses a wide range of responsibilities. In addition to leading and managing a team of talented engineers, the role involves overseeing projects from start to finish, making sure

everything runs smoothly and meets high standards. As an engineering manager, you'll also have the opportunity to shape the team by actively participating in recruiting and hiring. One of the key aspects of the role is setting ambitious goals and measuring progress through OKRs (Objectives and Key Results), providing a clear roadmap for success. Moreover, fostering collaboration between different departments and teams is vital to ensure a seamless and efficient workflow. It's a multifaceted role that combines the best of leadership, project management, and team building, making each day full of diverse and engaging challenges.

This page will capture some of the responsibilities and resources for the Create:Code Review Backend Engineering Manager.

People Management

Talent Assessments

Talent assessments happen on a regular basis.

Milestone Planning

Milestone Schedule

Here are the activities done by the engineering manager throughout the course of a milestone.

The release is on the 3rd Thursday of every month. We use a cutoff date of the Friday before that to get any work completed.

You can see more about the release process [here](#).

On the first of the month a new planning issue is automatically created for the next milestone. Create threads for the various projects / topics (e.g. Rapid Diffs, CLI, Bugs) and tag the backend team to get ideas for issues to include in the upcoming milestone planning.

Planning week activities

The last week of the milestone is planning week for the next milestone. Here are the tasks to do during this week and the following one:

| Day | Action |

|-----|-----|

| Mon | Make sure we have a capacity estimate for the next milestone. Check with team members to make sure all PTO has been entered. Use the capacity planning spreadsheet and add the capacity estimate to the planning issue. |

| Mon | Estimate slippage by asking team members which issues are likely/sure to slip, and the total weight expected to complete the work in the next milestone. |

| Mon | Have an initial planning meeting with the PM and frontend EM for Code Review to align on priorities for the milestone and set initial expectations on slippage. |

| Thu | Have a final planning meeting with the PM and frontend EM for Code Review to pick issues for the milestone that fit within the available capacity. |

| Fri | Assign issues to team members based on capacity. |

| Fri | Label all issues with the Deliverable and workflow::ready for development labels. |

| Fri | Post a comment with an overview of all assigned issues in the planning issue. |

-	Milestone ends
Mon	Calculate the percentage breakdown of features, bugs, and maintenance and add it to the current planning issue.
Tue	Review all open issues from the previous milestone, move them to the appropriate milestone.

Capacity planning

We use a lightweight system of issue weighting to help with capacity planning. These weights help us ensure that the amount of scheduled work in a cycle is reasonable, both for the team as a whole and for each individual. The "weight budget" for a given cycle is determined based on the team's recent output, as well as the upcoming availability of each engineer.

Since things take longer than you think, it's OK if an issue takes longer than the weight indicates. The weights are intended to be used in aggregate, and what takes one person a day might take another person a week, depending on their level of background knowledge about the issue. That's explicitly OK and expected. We should strive to be accurate, but understand that they are estimates! Change the weight if it is not accurate or if the issue becomes harder than originally expected. Leave a comment indicating why the weight was changed and tag your EM so that we can better understand weighting and continue to improve.

Weights

The weights we use are:

```
{{% include "includes/engineering/create/weighttable.md" %}}
```

Anything larger than 5 should be broken down if possible.

Security issues are typically weighted one level higher than they would normally appear from the table above. This is to account for the extra rigor of the patch release process.

In particular, the fix usually needs more-careful consideration, and must also be backported across several releases.

Availability

Approximately 5-10 business days before the start of a new release, the EM will begin determining how "available" the team will be. This is recorded in the capacity planning spreadsheet for transparency. Some of the things that will be taken into account when determining availability are:

- Upcoming training
- Upcoming time off / holidays
- Upcoming on-call slots
- Potential time spent on another teams deliverables

Availability is a percentage calculated by (work days available / work days in release) 100.

All individual contributors start with a "weight budget" of 7, meaning they are capable (based on historical data) of completing a maximum number of issues worth 7 weight points total. Then, based on their availability percentage, weight budgets are reduced individually. For example, if you are 70% available, your weight budget becomes 5.

Product will prioritize issues based on the teams total weight budget.

Project Management

Service Level Objectives Monitoring

The Engineering Manager will take proactive steps to avoid having reported bugs to go past the target duration defined in Severity SLOs.

The Bug Triage Board and Triage Reports can be used to review all open bugs.

Priority Issues

There are some classifications of issues that need immediate attention. These could be reliability, security, or high-priority issues. The Engineering Manager needs to watch for these issues on a regular basis, typically multiple times per week.

The Code Review Priority Issue Board is a way to see all these issues in one place.

SECTION: engineering/devops/dev/create/engineering-managers/_index.md

Meet the Engineering Managers

Image	Name	Title	Team	LinkedIn
-----	----	-----	----	-----
		Darva Satcher	Director of Engineering	Create Stage LinkedIn
		Adebayo Adesanya	Fullstack Engineering Manager	Remote Development LinkedIn
		André Luís	Senior Engineering Manager	Code Review & Source Code
LinkedIn				
		François Rosé	Fullstack Engineering Manager	Code Review LinkedIn

SECTION: engineering/devops/dev/create/engineering-managers/books.md

What books do we recommend for Engineering Managers?

New Managers

- The First 90 Days, Updated and Expanded: Proven Strategies for Getting Up to Speed Faster and Smarter

Recognition of Team Members

- Thanks for the Feedback: The Science and Art of Receiving Feedback Well

Time Management

- Make Time: How to Focus on What Matters Every Day
- The Surprising Science of Meetings: How You Can Lead Your Team to Peak Performance

Results

- The 4 Disciplines of Execution: Achieving Your Wildly Important Goals
- Getting Things Done: The Art of Stress-Free Productivity
- Atomic Habits: An Easy & Proven Way to Build Good Habits & Break Bad Ones
- The Power of Habit: Why We Do What We Do in Life and Business
- Failing Forward

Diversity Inclusion & Belonging

- Wisdom at Work
- The Culture Map: Breaking Through the Invisible Boundaries of Global Business

Coaching

- Multipliers
- Challenging Coaching: Going Beyond Traditional Coaching to Face the FACTS
- The Coaching Habit: Say Less, Ask More & Change the Way You Lead Forever

Leadership

- Who Moved My Cheese?: An A-Mazing Way to Deal with Change in Your Work and in Your Life
- Accelerate: Building and Scaling High Performing Technology Organizations
- Dare to Lead: Brave Work. Tough Conversations. Whole Hearts.
- Turn the Ship Around!: A True Story of Turning Followers into Leaders
- The 21 Irrefutable Laws of Leadership: Follow Them and People Will Follow You

SECTION: engineering/devops/dev/create/engineering-managers/live.md

The Create Engineering Managers lead by example. This page shares how the EMs live GitLab values.

Collaboration

- Bi Weekly EM Meetings
- Quarterly meetings with Stable Counterparts
- Recognition of team members through Discretionary Bonuses, Sync Team Meetings, Thanks Channel or Individual Team Channels

Results

- Development and implementation of Quarterly OKRs
- Planning Sprints with Product Managers
- Manage Employee Development
- Propose Cross Team or Cross stage Initiatives
- Strive for 70-80% Create Say / Do Metrics
- Participation in Working Groups
- Regularly Review Dashboards
- Host Team Retrospectives
- Continuously refine and improve team processes based on feedback from Retrospectives, Skip Level Meetings, 1-1's, 360 Feedback and Employee Engagement Surveys
- Support Team Members in their Employee Development

Efficiency

Communications

- Weekly Announcements to distribute communications from a centralized location
- Issue Board for tracking Engineering Manager action items
- Team Slack Channels
- Team Video Weekly Updates

Meetings

- Pre-populated Meeting Agendas
- Always prepare agendas as far as possible in advance and ask the attendees to review and fill out as much as they can before the meeting
- Default meetings to a maximum of 30 minutes
- If you find you must have a 1 hour meeting determine if you can break it into two sections, Verbalize and Read Only
- Pre-populate the agenda and ask the attendee to complete as much of the agenda as possible and when you are actually in the meeting, skip over the sections that have already been resolved

Time Management

- Audit your work schedule by determining where you are spending the most time
- Analyze your work schedule and determine if there are opportunities for delegation
- Use the calendar to color code your activity categories to better visualize where you are spending your time (e.g: Focus Time, 1:1s, Coffee Chats, Team Sync Meetings, Meeting Prep, Focused Software Development, etc)
- Compare where you are spending your time to your Job Family and to the Engineering Career Framework and ensure you are working on the right things

Delegation

- Delegation is an Empowering gift that you can give to a team member
- Delegating empowers other team members to take on new responsibilities and opportunities for professional growth

- Delegation Activity Worksheet is a tool Engineering Managers can use periodically to audit if there are opportunities for delegation

Calendar Best Practices

- Schedule Focus time in a minimum of 1 hour increments
- If you have routine activities you perform daily schedule them on your calendar
- If a regular deadline comes up that you keep forgetting about, create a calendar reminder
- Do not schedule more than 2 hours worth